

AW87XXX Android Driver(SPRD)

版本： V3. 6

时间： 2023-12

contents

1. INFORMATION	3
2. DRIVER PORTING	3
2.1 AW87XXX DRIVER PORTING	3
2.1.1 DTS CONFIGURATION	3
2.1.2 DRIVER CONFIGURATION	4
2.1.3 BIN CONFIGURATION	5
2.2 PLATFORM DRIVER CONFIGURATION	6
2.2.1 KCONTROL REGISTRATION	6
2.2.2 PA SCENE CONFIGURATION	7
2.3 DRIVE VERIFICATION	8
3. DEBUG INTERFACE	9
3.1 DEVICE NODE	9
4. APPENDIXX	11
4.1 FAQ	11
4.1.1 BIN LOADING FAILED	11
4.1.2 WHITE NOISE APPEARS AFTER TRANSPLANTING THE AWINIC ALGORITHM	11

1. INFORMATION

Driver File	aw87xxx.c, aw87xxx.h, aw87xxx_device.c, aw87xxx_device.h, aw87xxx_monitor.c, aw87xxx_monitor.h, aw87xxx_dsp.c, aw87xxx_dsp.h, aw87xxx_acf_bin.c, aw87xxx_acf_bin.h, aw87xxx_log.h, aw87xxx_bin_parse.c, aw87xxx_pid_39_reg.h, aw87xxx_pid_59_3x9_reg.h, aw87xxx_pid_59_5x9_reg.h, aw87xxx_pid_5a_reg.h, aw87xxx_pid_18_reg.h, aw87xxx_pid_9b_reg.h, aw87xxx_pid_76_reg.h, aw87xxx_pid_60_reg.h, aw87xxx_pid_c1_reg.h, aw87xxx_pid_c2_reg.h
Smart PA	aw87319, aw87329, aw87339, aw87349, aw87359, aw87389, aw87509, aw87519, aw87529, aw87539, aw87549, aw87559, aw87569, aw87579, aw81509, aw87579G aw87390, aw87360, aw87401, aw87390G aw87358, aw87560, aw87550 aw87391, aw87392 aw87565, aw87566, aw81564, aw87567, aw87568
I ² C Address	aw87358:0x5C others:0x58, 0x59, 0x5A, 0x5B

2. DRIVER PORTING

2.1 AW87XXX Driver Porting

2.1.1 DTS Configuration

AW87359/AW87389/AW87549/AW87390/AW87360/AW87401/AW87390G/AW87391/AW87392 products have no reset pin, so reset GPIO cannot be configured.

Product information sheet

Products	ChipID	reset-pin
AW87319	0x9B	Yes
AW87329/AW87339/AW87349	0x39	Yes
AW87519/AW87529	0x59	Yes
AW87359/AW87389	0x59	No
AW87549	0x5A	No
AW87559/AW87569/AW87579/AW81509/AW87579G	0x5A	Yes
AW87390/AW87360/AW87401/AW87390G	0x76	No
AW87358	0x18	Yes
AW87560/AW87550	0x60	Yes
AW87391/AW87392	0xC1	No
AW87565/AW87566/AW81564/AW87567/AW87568	0xC2	Yes

Mono PA Configuration

```
&i2c_x {                                     /*x: I2C BUS*/
    aw87xxx_pa@58 {                         /*0x58: I2C Address*/
        compatible = "awinic,aw87xxx_pa";
        reg = <0x58>;
        /*
AW87359/AW87389/AW87549/AW87390/AW87360/AW87401/AW87390G/AW87391/AW87392 products
reset GPIO cannot be configured */
        reset-gpio = <&ap_gpio 106 0>; /*Take GPIO 106 as an example */
        dev_index = < 0 >;                /*The first PA configured as 0*/
        status = "okay";
    };
};
```

Multi PA configuration

```
&i2c_x {
    aw87xxx_pa@58 {
        compatible = "awinic,aw87xxx_pa";
        reg = <0x58>;
        /*
AW87359/AW87389/AW87549/AW87390/AW87360/AW87401/AW87390G/AW87391/AW87392 products
reset GPIO cannot be configured */
        reset-gpio = <&ap_gpio 106 0>;
        dev_index = < 0 >;                /*The first PA configured as 0*/
        status = "okay";
    };
    aw87xxx_pa@59 {
        compatible = "awinic,aw87xxx_pa";
        reg = <0x59>;
        /*
AW87359/AW87389/AW87549/AW87390/AW87360/AW87401/AW87390G/AW87391/AW87392 products
reset GPIO cannot be configured */
        reset-gpio = <&ap_gpio 107 0>;
        dev_index = < 1 >;                /* The Second PA configured as 1*/
        status = "okay";
    };
};

/*The above takes double PA as an example, dev_index of multiple PA increases in
turn. Refer to single PA configuration for other attributes*/
```

2.1.2 Driver Configuration

defconfig Configuration:

```
CONFIG_SND_SOC_AW87XXX=y
```

Create aw87xxx directory in the kernel codecs directory and add driver files:

```
aw87xxx.c, aw87xxx_device.c, aw87xxx_monitor.c, aw87xxx_dsp.c, aw87xxx_acf_bin.c,
aw87xxx_bin_parse.c aw87xxx.h, aw87xxx_device.h, aw87xxx_monitor.h, aw87xxx_dsp.h,
aw87xxx_acf_bin.h, aw87xxx_log.h, aw87xxx_pid_39_reg.h, aw87xxx_pid_59_3x9_reg.h,
aw87xxx_pid_59_5x9_reg.h, aw87xxx_pid_5a_reg.h, aw87xxx_pid_18_reg.h,
aw87xxx_pid_9b_reg.h, aw87xxx_pid_76_reg.h, aw87xxx_pid_60_reg.h, aw87xxx_pid_c1_reg.h,
aw87xxx_pid_c2_reg.h
```

Kconfig Configuration:

```
config SND_SOC_AW87XXX
    tristate "SoC Audio for awinic AW87XXX Smart K PA"
    depends on I2C
help
    This option enables support for AW87XXX Smart K PA.
```

Makefile Configuration:

```
obj-$(CONFIG_SND_SOC_AW87XXX) += aw87xxx/aw87xxx.o
aw87xxx/aw87xxx_device.o aw87xxx/aw87xxx_monitor.o
aw87xxx/aw87xxx_bin_parse.o aw87xxx/aw87xxx_dsp.o
aw87xxx/aw87xxx_acf_bin.o
```

2.1.3 BIN Configuration

PA needs to configure register parameters to work normally. The configuration steps of PA bin file are as follows:

Compile Configuration

Add the bin file compilation option at the corresponding location of the platform:

```
PRODUCT_COPY_FILES += \
    xxx/aw87xxx_acf.bin:$(TARGET_COPY_OUT_VENDOR)/firmware/aw87xxx_acf.bin

/*xxx means Platform path */
```

Path Configuration

Add the directory of bin file in the firmware_class.c, the general directory is **vendor / firmware**

```
static const char * const fw_path[] = {
    fw_path_para,
    "/vendor/firmware",
    "/lib/firmware/updates/" UTS_RELEASE,
    "/lib/firmware/updates",
    "/lib/firmware/" UTS_RELEASE,
    "/lib/firmware"
};
```

```
/* Note: In the debugging stage, you can directly push the bin file to / vendor
/ firmware / */
```

Bin selection

Taking aw87390 as an example, the bin file contains the music / receiver / off scene:

```
aw87xxx_driver      /* driver root directory */
├── config           /* bin file directory */
│   └── aw87390
│       ├── mono
│       │   └── aw87xxx_acf.bin    /* mono configuration*/
│       └── stereo
│           └── aw87xxx_acf.bin    /* stereo configuration*/
└── monitor_bin_guide /* monitor bin directory */
    ├── aw87xxx_monitor.bin
    ├── aw87xxx_monitor_bin_guide.docx
    ├── aw87xxx_monitor_bin_guide.pdf
    ├── monitor.exe
    └── monitor_params.txt
```

2.2 Platform Driver Configuration

2.2.1 Kcontrol Registration

aw87xxx_codec registration:

```
/*Take the UMS512 platform as an example*/
@@ -195,6 +195,8 @@ static int sprd_codec_get_ctrl(struct snd_soc_codec *codec,
char *name);
static void sprd_codec_psg_state_init(struct snd_soc_codec *codec);
static void sprd_codec_psg_state_exit(struct snd_soc_codec *codec);
/*add interface*/
+ extern int aw87xxx_add_codec_controls(void *codec);
+
static unsigned long sprd_codec_dp_base;

enum {
@@ -3955,6 +3957,13 @@ static int sprd_codec_soc_probe(struct snd_soc_codec
*codec)
    return ret;
}

/*add controls registration */
+ ret = aw87xxx_add_codec_controls((void*) codec);
+ if(ret < 0) {
+     pr_err("add control failed\n");
+ }
+
+ return 0;
}
```

2.2.2 PA Scene Configuration

```
/* Take the UMS512 platform as an example, modify ums512-mach.dtsi */
& sound_vbc_v4_sprd_codec {
    status = "okay";

+   /*<0 0 4 0> means ext_control_type, hook selection, mode, gpio share
    Separately*/
+   sprd,spk-ext-pa-info = <0 0 4 0>;
+   sprd,spk-ext-pa-gpio = <&ap_gpio 156 0>;
};

/*hook selection represents loaction of aw87xxx interface in the following
ext_hook_arr array*/
```

```
/* Take the UMS512 platform as an example, modify sprd-asoc-card-utils-hook.c */
@@ -106,6 +106,32 @@ static int ext_debug_sysfs_init(void)
    return ret;
}

+#ifdef CONFIG_SND_SOC_AW87XXX
+extern int aw87xxx_set_profile(int dev_index, char *profile);
+
+static char *aw_profile[] = {"Music", "Off"}; /*scene aw87xxx_acf.bin*/
+enum aw87xxx_dev_index {
+    AW_DEV_0 = 0,
+};
+
+static int hook_spk_aw87xxx(int id, int on)
+{
+    pr_info("%s id: %d, on: %d\n", __func__, id, on);
+    if(on)
+        aw87xxx_set_profile(AW_DEV_0, aw_profile[0]);
+    else
+        aw87xxx_set_profile(AW_DEV_0, aw_profile[1]);
+    return HOOK_OK;
+}
+#else
+static void hook_gpio_pulse_control(unsigned int gpio, unsigned int mode)
+{
+    int i = 1;
+@@ -161,9 +187,14 @@ static int hook_general_spk(int id, int on)
+
+    return HOOK_OK;
+}
+#endif

+static struct sprd_asoc_ext_hook_map ext_hook_arr[] = {
+#ifdef CONFIG_SND_SOC_AW87XXX:
+    {"aw87xx", hook_spk_aw87xxx, EN_LEVEL},
+#else
```

```
        {"general_speaker", hook_general_spk, EN_LEVEL},
+ #endif
};
```

2.3 Drive Verification

Take aw87560 single PA registration as an example:

1) I2C communication successful:

```
[Awinic] aw87xxx_pa_init:driver version: Dv2.4.0.5
[Awinic] [0-0058]aw87xxx_malloc_init: struct aw87xxx devm_kzalloc and init down
[Awinic] [0-0058]aw87xxx_dtsi_parse: parse dev_index=[0]
[Awinic] [0-0058]aw87xxx_dtsi_parse: reset gpio[1160] parse succeed
[Awinic] [0-0058]aw87xxx_device_parse_port_id_dt: rx-port-id: 0xb030
[Awinic] [0-0058]aw87xxx_device_parse_topo_id_dt: rx-topo-id: 0x1000ff01
[Awinic] [0-0058]aw87xxx_dev_hw_pwr_ctrl: hw power on
[Awinic] [0-0058]aw87xxx_dev_get_chipid: read chipid[0x60] succeed
[Awinic] [0-0058]aw_dev_chip_init: product is pid_60 class
```

2) PA Bin loaded successfully:

```
[Awinic] [0-0058]aw_get_prof_count_v_0_0_0_1: get profile count=[3]
[Awinic] [0-0058]aw_get_vaild_prof_v_0_0_0_1: product=[aw87560]----profile=[Music]
[Awinic] [0-0058]aw_get_vaild_prof_v_0_0_0_1: product=[aw87560]----profile=[Receiver]
[Awinic] [0-0058]aw_set_prof_off_info_v_0_0_0_1: product=[aw87560]----profile=[Off]
[Awinic] [0-0058]aw_get_vaild_prof_v_0_0_0_1: get vaild profile succeed
[Awinic] [0-0058]aw_set_prof_name_list_v_0_0_0_1: index=[0], profile_name=[Music]
[Awinic] [0-0058]aw_set_prof_name_list_v_0_0_0_1: index=[1], profile_name=[Receiver]
[Awinic] [0-0058]aw_set_prof_name_list_v_0_0_0_1: index=[2], profile_name=[Off]
[Awinic] [0-0058]aw_parse_acf_v_0_0_0_1: acf parse success
[Awinic] [0-0058]aw87xxx_init_default_prof: init profile name [Off]
[Awinic] [0-0058]aw87xxx_fw_load: acf parse succeed
[Awinic] [0-0059]aw87xxx_fw_load: enter
```

3) Kcontrol registration successful:

```
[Awinic] [0-0058]aw87xxx_kcontrol_dynamic_create: enter
[Awinic] [0-0058]aw87xxx_kcontrol_dynamic_create: add codec controls[aw87xxx_profile_switch_0,aw87xxx_vmax_get_0,aw87xxx_monitor_switch_0]
[Awinic] [0-0058]aw87xxx_public_kcontrol_create: enter
[Awinic] [0-0058]aw87xxx_public_kcontrol_create: add public codec controls[aw87xxx_spin_switch]
[Awinic] [0-0058]aw87xxx_profile_switch_get: current profile:[Off]
```

4) Set profile successful:

```
kona:/sys/bus/i2c/drivers/aw87xxx_pa/0-0058 # cat profile
Music
Receiver
>Off
kona:/sys/bus/i2c/drivers/aw87xxx_pa/0-0058 # echo Music > profile
[Awinic] [0-0058]aw87xxx_attr_get_profile: current profile:[Off]
[Awinic] [0-0058]aw87xxx_attr_set_profile: set profile [Music]
[Awinic] [0-0058]aw87xxx_update_profile: load profile[Music] enter
[Awinic] [0-0058]aw87xxx_monitor_stop: enter
[Awinic] [0-0058]aw87xxx_power_on: enter
[Awinic] [0-0058]aw87xxx_acf_get_prof_desc_form_name: enter
[Awinic] [0-0058]aw87xxx_acf_get_prof_desc_form_name: get prof desc down
[Awinic] [0-0058]aw87xxx_power_on: get profile[Music] data len [84]
```

5) Play music successful:

```
[Awinic] aw87xxx_set_profile_by_id:aw87xxx, dev_index[0] set profile[Music] by id[1]
[Awinic] [0-0058]aw87xxx_set_profile: set dev_index = 0, profile = Music
[Awinic] [0-0058]aw87xxx_update_profile: load profile[Music] enter
[Awinic] [0-0058]aw87xxx_monitor_stop: enter
[Awinic] [0-0058]aw87xxx_power_on: enter
[Awinic] [0-0058]aw87xxx_acf_get_prof_desc_form_name: enter
[Awinic] [0-0058]aw87xxx_acf_get_prof_desc_form_name: get prof desc down
[Awinic] [0-0058]aw87xxx_power_on: get profile[Music] data len [84]
[Awinic] [0-0058]aw87xxx_dev_hw_pwr_ctrl: hw power on
[Awinic] [0-0058]aw87xxx_dev_reg_update: reg=0x76, val = 0x92
[Awinic] [0-0058]aw87xxx_dev_reg_update: reg=0x7d, val = 0x7a
[Awinic] [0-0058]aw87xxx_dev_reg_update: reg=0x7c, val = 0xc1
```

3. DEBUG INTERFACE

3.1 Device Node

AW87XXX Driver will create multiple device node files with different functions. The path is sys/bus/i2c/driver/aw87xxx_smartpa/*-00xx, where * is the i2c bus number and xx is the i2c address.

reg

Node name	reg
Description	read and write all register value of aw87xxx
Instruction	read register value: echo reg_addr > rw (Hexadecimal operation) cat rw write register value: echo reg_addr reg_data > rw (Hexadecimal operation)
Example	cat reg (get all the values of the register with read permission) echo 0x01 0x07 > reg (write the value of 0x07 to the register of 0x01)

profile

Node name	profile
Description	scene switch
Instruction	View the current scene and switchable scenes: cat profile Select scene: echo xxxx > profile (xxxx means scene name)
Example	cat profile echo "Music" > profile (Select Music scene) echo "Off" > profile (Select Off scene)

hwen

Node name	hwen
Description	Control PA status
Instruction	cat hwen (get PA status)

	echo 1 > hwen	(aw87xxx power on)
	echo 0 > hwen	(aw87xxx power down)

esd_enable

Node name	esd_enable	
Description	Esd function switch	
Instruction	cat esd_enable	(Get status of ESD function)
	echo true > esd_enable	(Enable the ESD function)
	echo false > esd_enable	(Disable the ESD function)

vmax

Node name	vmax	
Description	set vmax to dsp and get the current vmax value of dsp	
Instruction	cat vmax	(Get the value of the current vmax)
	echo N > vmax	(Send calculated vmax value)
	echo 0xffff95f7e > vmax	(Send 0xffff95f7e value to dsp)

vbat

Node name	vbat	
Description	set the current power and get the real-time power value	
Instruction	cat vbat	(Get real-time value)
	echo capacity > vbat	(Set vbat)
	echo 0 > vbat	(Cancel input vbat before)
Example	echo 50 > vbat	(Set the current power to 50%)

monitor

Node name	monitor	
Description	Enable or disable monitor function	
Instruction	cat monitor	(get monitor switch status)
	echo 0 > monitor	(disable monitor)
	echo 1 > monitor	(enable monitor)

monitor_count

Node name	monitor_count	
Description	Set the monitor interval for Vmax	
Instruction	cat monitor_count	(get monitor interval)

	echo 5 > monitor_count	(Set the monitor interval to 5)
--	------------------------	---------------------------------

monitor_time

Node name	monitor_time	
Description	Set monitor interval time(ms)	
Instruction	cat monitor_time	(get monitor interval time)
	echo N > monitor_time	(set monitor interval time)

rx

Node name	rx	
Description	Set or get dsp status	
Instruction	cat rx	(get rx status of dsp)
	echo 1 > rx	(enable dsp rx module)
	echo 0 > rx	(disable dsp rx module)

monitor_update

Node name	monitor_update	
Description	update monitor parameter configuration	
Instruction	echo 1 > monitor_update	(update monitor parameter)

4. APPENDIX

4.1 FAQ

4.1.1 BIN loading Failed

1) Reason:

Aw87xxx driver needs load bin files, but some platforms fail to load bin files due to SELinux permissions.

2) Solution:

In the corresponding file named kernel.te add missing permissions. Format: allow kernel_vendor_file:file {getattr read}

4.1.2 White noise appears after transplanting the AWINIC algorithm

If there is regular white noise playback during PA operation (audio source 10s ->white noise 3s->audio source 10s ->white noise 3s->.....)

The above phenomenon indicates that the awinic algorithm authentication has failed. Please enable the corresponding function in the driver to recompile.

```
diff --git a/aw87xxx_device.h b/aw87xxx_device.h
index 6234a73..ad62b2c 100644
--- a/aw87xxx_device.h
+++ b/aw87xxx_device.h
@@ -64,7 +64,7 @@
     *****/
 struct aw_device;

-/*#define AW_ALGO_AUTH_DSP*/
+#define AW_ALGO_AUTH_DSP
extern int g_algo_auth_st;

enum AW_ALGO_AUTH_MODE {
```